Reviewer Pack — Schur-Weyl Dimension Gap in 3-CNF Permanent vs Determinant S…

Entry 390e4c06544a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 12:46:32 UTC

Schur-Weyl Dimension Gap in 3-CNF Permanent vs Determinant Shapes

Entry ID: `390e4c06544a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 12:46:32 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): 3-CNF SAT Instances

Statement:

> For random 3-CNF instances with n variables and m clauses, the number of standard Young tableaux of shape (m, m, …, m) (m rows of length 3) is ≥ $2^{\{n}2/4\}$ times the number for shape (n, n-1, …, 1).

Rationale (proposer's reasoning):

> Schur-Weyl duality links polynomial representations to symmetric group actions; the dimension gap between rectangular and staircase Young diagrams mirrors the algebraic complexity of permanent vs determinant. Exponential separation in SYT counts could imply inherent structural differences in their representation-theoretic decompositions.

Taxonomy category: `BOUNDED_ARITHMETIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c287fd5a5c1840c5`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), random.randint(1, n), random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def hook_length_formula(shape):
        n = len(shape)
        total = 1
        for i in range(n):
            for j in range(i + 1):
                total *= (shape[i] - j) * (n - i - j)
        for i in range(1, n):
            for j in range(i):
                total //= (i - j) * (j + 1)
        return math.factorial(n) // total

    def count_syt(shape):
        m = len(shape)
        if shape == [3] * m:
            cnf = generate_3cnf(m, m)
            # Count SYT for permanent shape
            syt_count = 0
            for perm in itertools.permutations(range(1, m + 1)):
                valid = True
                for i in range(m):
                    if (perm[i] % 3 == 0 and perm[(i + 1) % m] % 3 != 0) or \
                       (perm[i] % 3 == 1 and perm[(i + 2) % m] % 3 != 0) or \
                       (perm[i] % 3 == 2 and perm[(i + 4) % m] % 3 != 0):
                        valid = False
                        break
                if valid:
```

```
                    syt_count += 1
            return syt_count
        else:
            # Count SYT for determinant shape
            n = len(shape)
            total = 0
            for perm in itertools.permutations(range(1, n + 1)):
                valid = True
                for i in range(n):
                    if (perm[i] % n == 0 and perm[(i + 1) % n] % n != 0) or \
                       (perm[i] % n == 1 and perm[(i + 2) % n] % n != 0) or \
                       (perm[i] % n == 2 and perm[(i + 3) % n] % n != 0):
                        valid = False
                        break
                if valid:
                    total += 1
            return total

    def is_prime(num):
        if num < 2:
            return False
        for i in range(2, int(math.sqrt(num)) + 1):
            if num % i == 0:
                return False
        return True

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = n // 3
        permanent_shape = [3] * m
        determinant_shape = list(range(n, 0, -1))

        permanent_syt_count = count_syt(permanent_shape)
        determinant_syt_count = count_syt(determinant_shape)

        ratio = permanent_syt_count / (2 ** (n ** 2 / 4) * determinant_syt_count)

        results.append({
            "metric_name": "SYT Count Ratio",
            "metric_value": ratio,
            "instances_tested": 1,
            "conjecture_holds": ratio >= 1,
            "counterexample": "" if ratio >= 1 else f"3-CNF with n={n}, m={m} violates the conject
        })

    return {
        "metric_name": "SYT Count Ratio",
        "metric_value": sum(r["metric_value"] for r in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": all(r["conjecture_holds"] for r in results),
        "counterexample": next((r["counterexample"] for r in results if not r["conjecture_holds"])
    }

if __name__ == "__main__":
```

```
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c300c5c1.py", line 115, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c300c5c1.py", line 91, in run_trial
    ratio = permanent_syt_count / (2 ** (n ** 2 / 4) * determinant_syt_count)
            ~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with ZeroDivisionError, preventing validation of conjecture. Likely indicates code error rather than mathematical failure. | next: Debug the division-by-zero error in the test code's ratio calculation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_local	qwen3:8b	0	0	157702
2	propose	ollama_remote	qwen3:8b	0	0	33475
3	preregistration	ollama_remote	qwen3:8b	0	0	24026
4	novelty	ollama_remote	qwen3:8b	0	0	20846
5	novelty	ollama_remote	qwen3:8b	0	0	16500
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39404
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14182
8	judge	ollama_remote	qwen3:8b	0	0	17585

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 323720 ms total latency. Provider mix: {'ollama_local': 1, 'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/390e4c06544a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/390e4c06544a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/390e4c06544a.tar.gz` (if generated)